

Ethan Patterson
HW2- Problem 1

a.) Briefly, the solution is a sweep-line algorithm paired with a merge sort by y-coordinates ascending. More in depth, the solution requires that the horizontal bands between vertices be checked. This can be done by incrementing and decrementing a counter whenever an edge begins and ends. This is done by first running over the list of all vertices. As each vertex is listed in clockwise order, it can be assumed that an edge exists between vertices[i] and vertices[i+1]. As the sweep-line will begin at the lowest y-coordinate in the polygon and sweep to the top, the lowest point of the edge should correspond to incrementing the counter, and decrementing at the highest point of the edge. These two events correspond to entering and exiting the band occupied by an edge. As these are events, it follows that encountering the low and high can be organized into a list of events such that the lowest y-coordinate between vertices[i] and vertices[i+1] corresponds to a delta of +1, and the opposite for -1. Once events have been extracted from the edges, they must be sorted in order of y-coordinate ascending. This means that events will be ordered from bottom to top in order of when the sweeping line will encounter them. This can be done via a merge sort. Once sorted via merge sort, the list of events should be iterated through, with a counter recording the delta values at each event. As a reminder, these deltas correspond to entering and leaving edges. Thus, taking the maximum of this counter will mean that at a specific band of y-value, the algorithm will derive the point where there are the most edges existing in a band, i.e. the most intersections at a specific y-value.

b.) Pseudo Code:

```
for index in range(n-1):
    y1 = vertices[index]
    y2 = vertices[index+1]
    low = min(y1, y2)
    high = max(y1, y2)

    add event (low, +1)
    add event (high, -1)
merge_sort(events)
current = 0
best = 0
Pos = 0
Miny = events[0]
Maxy = events[events.size-1]
for(i = miny; i<=maxy; i++)
    for(j = pos; j<events.size; j++)
        if(events[j][0] >= i) break
        Current += events[j][1]
    best = max(best, current)
return best
```

- c.) Each line is two vertices. One can be considered the “top”, and one the “bottom”. We can assign the “top” the value of -1, and “bottom” the value of +1. Once we sort all vertices by their y values, we then check only the vertices below our y value in question. Any lines that are fully below add both the +1 and -1, cancelling out. Any bottoms below that are counted without a top, add the +1 only, which produces an accurate intersection count. Anything fully above is not touched, thus not counting.
- d.) This algorithm will produce a running time of $O(2n+n\log n)$, or simply $O(n\log n)$.
- e.) This estimate is derived from iterating over the vertices (which is of length n), then sorting the n events via merge sort ($n\log n$), and then once again iterating over all n events.